

Poster Presentation Script

Influence Maximization When Communication Is the Bottleneck

Yixin Chen

Bichi Zhang

Yaoqin Ye

CS240: Algorithm Design and Analysis, Spring 2026
June 18, 2026

How to Use This Script

This script is written for a poster-side presentation, not a slide talk. The recommended style is to point to the poster from left to right:

1. start from the research question at the top;
2. explain the optimization target, diffusion model, and submodularity on the left;
3. walk through the algorithmic story and distributed protocol in the center;
4. end with the large-scale evidence, communication tradeoff, and takeaways on the right.

The main version below is designed for about 4–5 minutes. A shorter version is included for a quick 90-second explanation.

1 Main Poster Talk: 4–5 Minutes

Opening

Hello, we are presenting our CS240 course project, *Influence Maximization When Communication Is the Bottleneck*. The broad problem is influence maximization: given a network, choose a small set of initial seed nodes so that information or behavior spreads to as many nodes as possible.

Our specific question is shown at the top of the poster: how much influence quality can we keep when expensive centralized seed selection is replaced by cached greedy evaluation or communication-limited local candidates?

So the project has two connected parts. First, we reproduce the classical algorithmic story: influence maximization under the Independent Cascade model is a monotone submodular maximization problem, so greedy has a principled approximation guarantee. Second, we ask what changes when the selection process is made more scalable by CELF lazy evaluation or by GreeDI-style distributed candidate sharing.

Problem and Model

On the left side of the poster, we define the optimization target. Given a directed graph $G = (V, E, p)$ and a seed budget k , we choose

$$S \subseteq V, \quad |S| \leq k,$$

to maximize the expected spread

$$\sigma(S).$$

In our implementation, $\sigma(S)$ is estimated by Monte Carlo simulation under the Independent Cascade model.

The Independent Cascade model is simple but expressive. When a node first becomes active, it gets one chance to activate each inactive out-neighbor. Each edge attempt succeeds independently with probability p_{uv} . For our experiments, we use the weighted-cascade rule,

$$p_{uv} = \frac{1}{\text{indegree}(v)}.$$

This means that nodes with many incoming neighbors are harder to activate from any single incoming edge.

The key theoretical point is submodularity. The expected influence function is monotone and submodular, meaning that the marginal gain of adding a new seed decreases as the current seed set grows:

$$\sigma(A \cup \{v\}) - \sigma(A) \geq \sigma(B \cup \{v\}) - \sigma(B) \quad \text{for } A \subseteq B.$$

This is why greedy is not just a heuristic here. For the exact objective, greedy has the classical $(1 - 1/e)$ guarantee. The practical bottleneck is that each marginal gain estimate requires many diffusion simulations.

Algorithms

The center of the poster shows the algorithmic story.

We start with simple structural baselines: random selection, weighted out-degree, and PageRank. These are cheap and help us understand how much of the performance comes from graph structure alone.

Then we implement centralized greedy. At each round, greedy evaluates the estimated spread of adding each remaining candidate node, and chooses the node with the largest marginal gain. This matches the submodular optimization theory, but it is expensive because it repeatedly calls the spread estimator.

Next is CELF, or Cost-Effective Lazy Forward. CELF uses a priority queue of cached marginal gains. Because submodularity tells us marginal gains can only go down as the seed set grows, old marginal gains are valid upper bounds. CELF therefore recomputes a node only when necessary. It targets the same greedy objective, but reduces redundant spread evaluations.

Finally, we study a GreeDI-style distributed method. The node set is randomly partitioned into m parts. Each local worker runs greedy over its local candidate nodes and sends at most q candidates to the coordinator. The coordinator takes the union of local candidates and runs a final greedy selection over that smaller pool.

This gives us the main distributed tradeoff: smaller q means lower communication, but it may miss useful candidates; larger q sends more candidates and gives the coordinator more choice, but it increases both communication and computation.

Experimental Setup

For experiments, we use synthetic graph families and one small real graph. The large preset uses Erdős–Rényi, Barabási–Albert, and Watts–Strogatz graphs with $n \in \{50, 100, 200\}$, plus the 34-node Karate Club graph. The seed budget is $k = 10$ for the large synthetic graphs and $k = 5$ for Karate.

For distributed experiments, we sweep

$$m \in \{2, 4, 8\}, \quad q \in \{k, 2k, 5k\}.$$

To make comparisons fair, after each algorithm selects its seed set, we re-evaluate the final seed set using a common evaluation seed for that graph. So the reported spread is not just the internal value used during selection.

Results

The right side of the poster summarizes the main results.

First, CELF is the clearest algorithmic improvement. On the large preset, centralized greedy uses about 1026.5 spread evaluations on average, while CELF uses 266.2. That is about a 74.1% reduction. Runtime also drops from about 4.67 seconds to 0.77 seconds on average, an 83.5% reduction. At the same time, the average spread stays very close to greedy: 40.05 for CELF versus 40.43 for greedy.

Second, weighted degree is surprisingly strong. Its average spread is 43.71, which is above our greedy result under fair re-evaluation. This does not contradict the greedy approximation theorem. The theorem is about exact greedy on the exact submodular objective. Our implementation uses Monte Carlo estimates, and the weighted-cascade probability rule naturally favors structurally central nodes. So this is a useful empirical lesson: on small course-scale graphs, graph structure and estimation noise both matter.

Third, the distributed method is competitive but not monotone in the communication budget. The best average distributed setting in the large preset is $m = 2, q = k$, which is essentially tied with centralized greedy after fair re-evaluation. However, increasing q does not automatically improve quality in this run. It clearly increases transmitted candidates and spread evaluations, but the quality still depends on partitions and Monte Carlo noise.

Conclusion

The main takeaway is that influence maximization gives a clean bridge from classical algorithms to modern network applications. Submodularity gives a principled greedy baseline. CELF turns the same theory into a practical speedup by avoiding unnecessary evaluations. Distributed candidate sharing changes the cost model: it can reduce global search, but it introduces communication limits, partition sensitivity, and more dependence on estimation stability.

So our project is not a production-scale influence maximization system. It is a course-scale reproduction of the main algorithmic mechanisms: greedy submodular optimization, lazy evaluation, and communication-constrained distributed selection.

2 Short Poster Talk: 90 Seconds

Our project studies influence maximization: choose k seed nodes in a network to maximize expected diffusion spread. Under the Independent Cascade model, the expected spread function is monotone and submodular, so greedy has the classical $(1 - 1/e)$ guarantee for the exact objective.

The practical problem is that greedy is expensive. Every round tries many candidate nodes, and each candidate requires Monte Carlo spread estimation. So we compare three levels of methods: simple structural baselines such as random, weighted degree, and PageRank; centralized greedy and CELF lazy greedy; and a GreedyDI-style distributed method where local workers send only a limited number of candidates to a coordinator.

Our clearest result is that CELF keeps almost the same spread as greedy but uses many fewer spread evaluations. On the large preset, evaluations drop from 1026.5 for greedy to 266.2 for CELF, and runtime drops from 4.67 seconds to 0.77 seconds on average.

For distributed candidate sharing, the main message is a tradeoff. Small candidate pools can be competitive, but larger q increases communication and computation and does not automatically improve quality under noisy Monte Carlo estimation.

Overall, the project shows how submodular optimization explains the greedy baseline, how CELF accelerates it, and how communication limits complicate influence maximization in distributed settings.

3 Common Questions and Suggested Answers

Q1. Why is influence maximization NP-hard?

Kempe, Kleinberg, and Tardos showed that influence maximization is NP-hard under standard diffusion models such as Independent Cascade and Linear Threshold. Intuitively, the seed set has to solve a combinatorial coverage-like problem over many possible diffusion outcomes. This is why we use greedy approximation rather than exact optimization.

Q2. What exactly is submodularity, and why does it matter here?

Submodularity means diminishing returns. If $A \subseteq B$, then adding a candidate node v to the smaller set A gives at least as much marginal gain as adding it to the larger set B . This matters because monotone submodular maximization under a cardinality constraint has a $(1 - 1/e)$ greedy approximation guarantee.

Q3. Does CELF change the greedy solution?

Conceptually, CELF is an acceleration of greedy, not a different objective. It uses cached marginal gains as upper bounds and recomputes only when needed. In exact deterministic evaluation, it can reproduce the greedy choices with far fewer evaluations. In our implementation, Monte Carlo randomness means the selected seed set can differ slightly, but the target remains the same greedy objective.

Q4. Why does weighted degree sometimes beat greedy in your table?

This is because our reported spread is a fair re-evaluation of final seed sets, while selection itself uses noisy Monte Carlo estimates. Also, the weighted-cascade probability rule makes structurally central nodes very strong. So this result does not violate the greedy theorem; it shows that on course-scale graphs, estimation noise and graph structure can make a cheap baseline look very strong.

Q5. Why use Monte Carlo simulation instead of exact spread?

Exact expected spread is computationally expensive because it averages over many possible diffusion outcomes. Monte Carlo simulation is simple and standard for course-scale experiments: run the diffusion process many times and average the final active set size.

Q6. Why did you use the weighted-cascade probability rule?

Weighted cascade, $p_{uv} = 1/\text{indegree}(v)$, is common in influence maximization experiments. It prevents high-indegree nodes from being too easy to activate by any one neighbor. It also gives a simple, reproducible way to assign probabilities to synthetic graphs.

Q7. What is the difference between greedy and PageRank or degree?

Degree and PageRank are structural heuristics. They rank nodes based on graph structure without simulating the diffusion objective for every possible seed set. Greedy directly estimates marginal influence spread at each step, so it is closer to the formal objective but much more expensive.

Q8. Why does distributed greedy evaluate spread on the full graph instead of only local subgraphs?

In our simplified model, each worker has a local candidate list, but influence can still spread globally. Evaluating on the full graph models the fact that a locally selected seed may activate nodes outside its partition. This keeps the experiment focused on candidate communication limits rather than on cutting off diffusion at partition boundaries.

Q9. Why does increasing q not always improve distributed quality?

Increasing q gives the coordinator more candidates, but the candidates are still selected using noisy Monte Carlo estimates. Larger q also changes the candidate pool and can introduce more redundant candidates. With small graphs and limited simulations, quality is not guaranteed to increase monotonically.

Q10. What are the main limitations of this project?

The biggest limitations are scale and randomness. We use course-scale graphs up to 200 synthetic nodes, not million-node real networks. We also use a modest number of Monte Carlo simulations for runtime reasons. A stronger study would use larger real datasets, more random seeds, confidence intervals, and a parallel implementation for the distributed workers.

Q11. Did you reproduce IMM, RIS, PaC-IM, or GreediRIS?

No. Those are treated as related work and motivation. Our implementation focuses on the core mechanisms that fit the course scope: IC simulation, centralized greedy, CELF lazy greedy, and GreeDI-style communication-limited candidate sharing.

Q12. What would be the next extension?

The most natural extension would be to average over multiple random seeds and report confidence intervals. Algorithmically, the next step would be an RR-set-based maximum coverage method, or a truly parallel implementation of the worker phase to measure real distributed speedup.

4 Backup Details

Key Numbers to Remember

Method	Spread	Ratio	Runtime	Evaluations
Greedy	40.43	1.00	4.67s	1026.5
CELF	40.05	0.99	0.77s	266.2
Weighted degree	43.71	1.07	0.01s	1.0

One-Sentence Summary

Our project shows that submodularity gives influence maximization a principled greedy baseline, CELF makes that baseline much cheaper, and distributed candidate sharing introduces a communication-quality tradeoff rather than a free speedup.